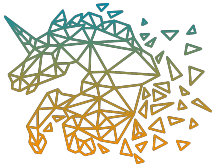
Documenting APIs



by Javier Rubio

@jrubr /javi-rubio /jrub



Software Crafters Barcelona

XII EDITION #SCBCN25

UNIXcorn

manfred

PegaSO

audiense

next digital

voxel an AMADEUS company

PonIA

540

basetis

codely

CODESAI

codurance craft at heart

holded

Rover

seQura

SirviendoCódigo

SOAP & WSDL

Strongly typed and strictly defined.

XML-based contracts: predictable, but verbose.

Great for enterprise integration, **terrible developer experience.**

Representational State Transfer (REST)

Promised simplicity and scalability.

But: no strict definition → chaos.

Misuse of HTTP verbs, inconsistent error handling.

Endless debates on plural nouns, pagination, and error codes.

GraphQL

Strict schema again, predictable responses.

Pros: Single endpoint, no overfetching.

Cons: Caching is hard, performance is tricky, steep learning curve.

Adoption stayed mostly within the companies that championed it.

gRPC

Built on HTTP/2 and Protocol Buffers.

Strongly typed, contract-first, and blazing fast.

Perfect for internal microservice communication, not public APIs.

Pros: Efficient, supports streaming, auto-generates client SDKs.

Cons: Binary format, not human-friendly, poor browser support.

Swagger (OpenAPI)

A standardized way to describe REST APIs in JSON or YAML.

Designed to bring structure and predictability to REST’s flexibility.

Auto-generates docs and a simple interactive UI.

Looked like the ultimate solution… but it wasn’t.

REST API conventions

OpenAPI has gaps.

You still need to manually document your API conventions:

Authentication, error handling, pagination, rate limiting, versioning.

OpenAPI fragmented ecosystem

Webhooks poorly supported until recent versions.

Disputes over **JSON-LD** for stricter specs.

Ecosystem fragmented with **proprietary extensions**.

New standards like **Arazzo** finally allow describing API workflows.

Tools, tools, tools… and pain

Endless tools, either laser-focused or “we-do-it-all”. None complete!

Expensive monthly subscriptions. Proprietary ecosystems.

Take a look at https://openapi.tools to understand…

Tools, tools, tools… and pain

Examples:

* **Postman** turned into a corporate monster.
* **Insomnia**, the alternative, monsterized too. Now: Yaak, Bruno.
* **Redoc** / **Redocly** split between open-source and enterprise features.
* Tooling even for generating the spec: linters, IDE plugins…

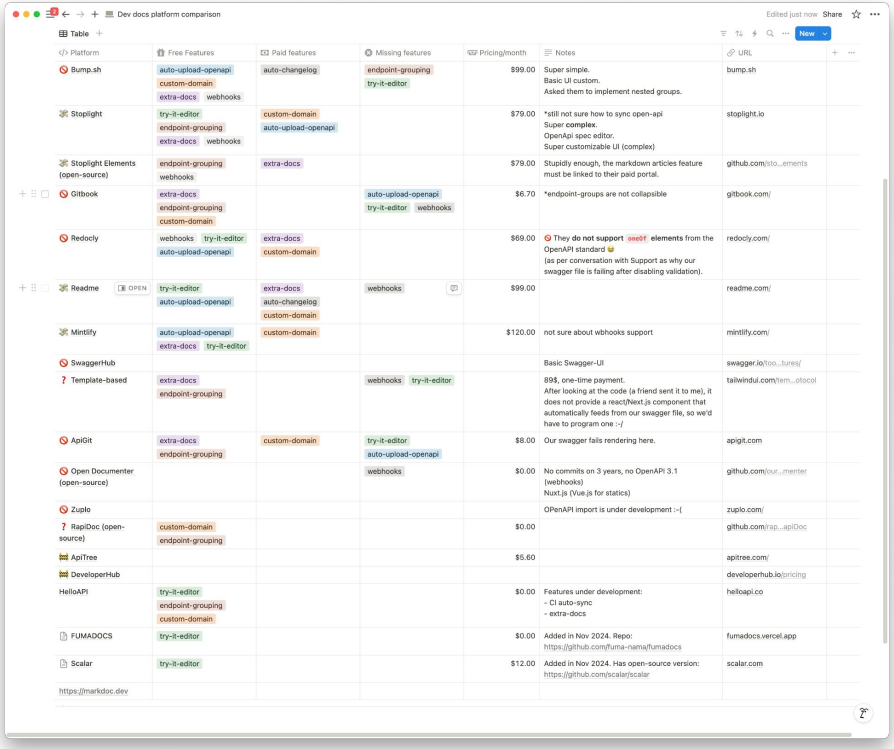
Tools, tools, tools… and pain

Some teams prefer static sites with Tailwind templates.

Flexible but high-maintenance.

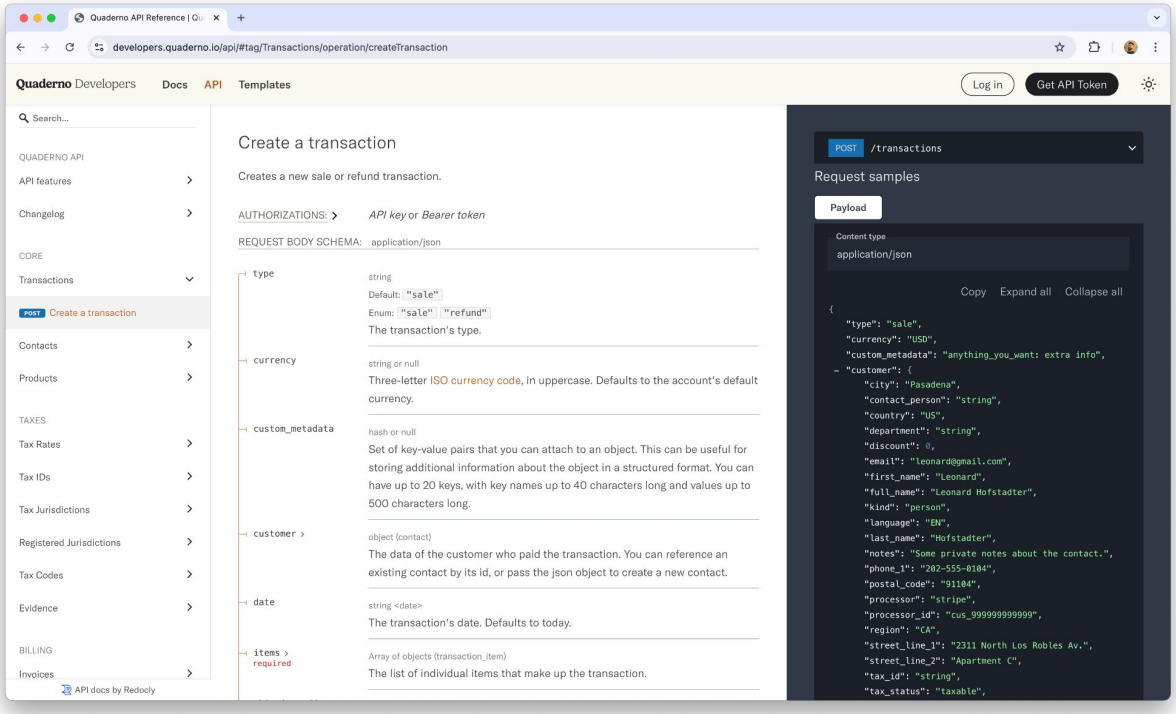
Generating and evolving your OpenAPI spec becomes a full-time job.

Choose tools that fit your workflow, not the other way around.



Investigating the tooling ecosystem before our launch

Example: Quaderno API reference



Docusarus (guides) + **redocusaurus** plugin (API reference)

My personal stand as a multidisciplinary professional

Technical Product Manager
Tech Writer
Dev Rel
CTO
Teacher
Android Dev
Technical Coach
Product Engineer
Organizational Consultant



Your API is a product

Treat it like one !



Documenting APIs from a product perspective

How would that be?



The closer to the code, the better.

Docs should ship with the code whenever the API changes**.**

Example from Quaderno:

We use Rswag (a Ruby on Rails gem) to auto-generate our OpenAPI spec from tests, open a PR in the docs repo for review, and then publish automatically once merged.

Know your users. Talk to them.

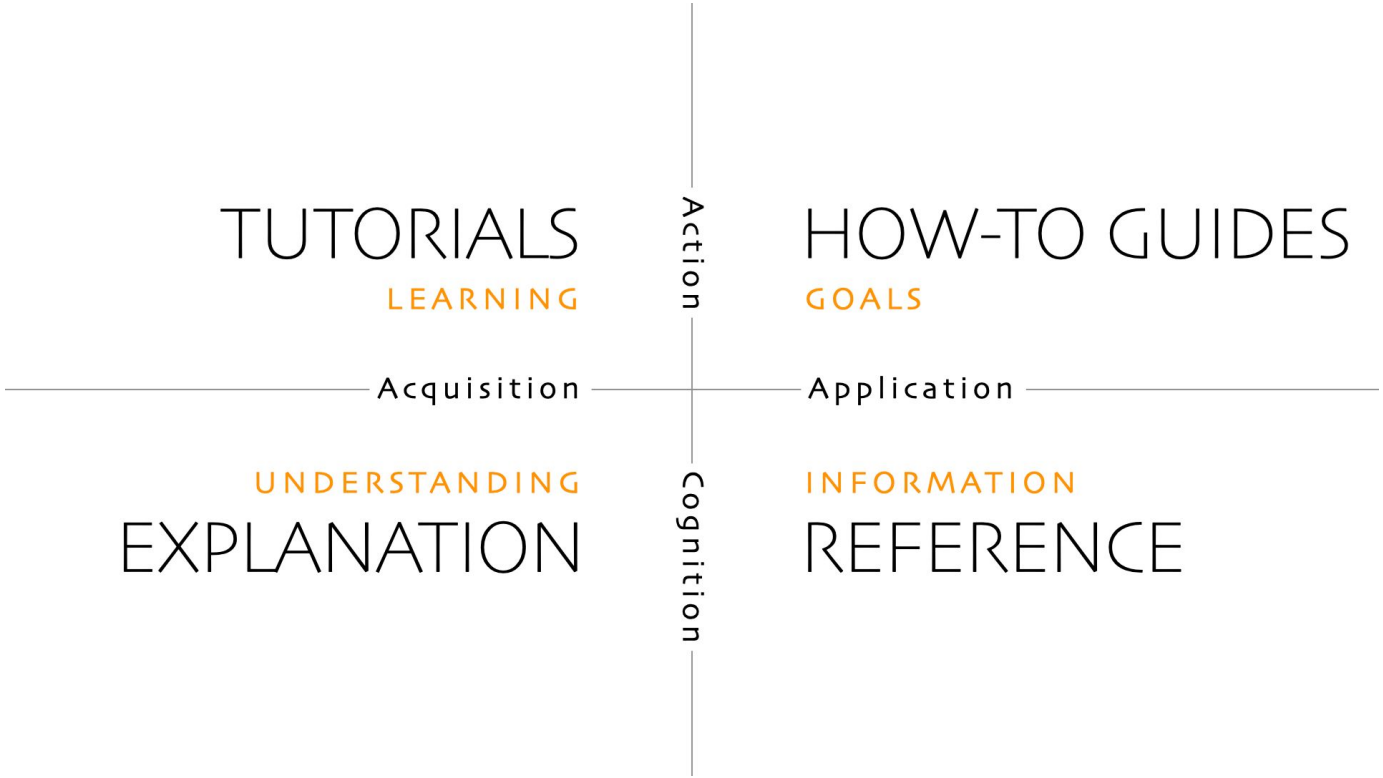
Ideally,

* Map your API user personas.

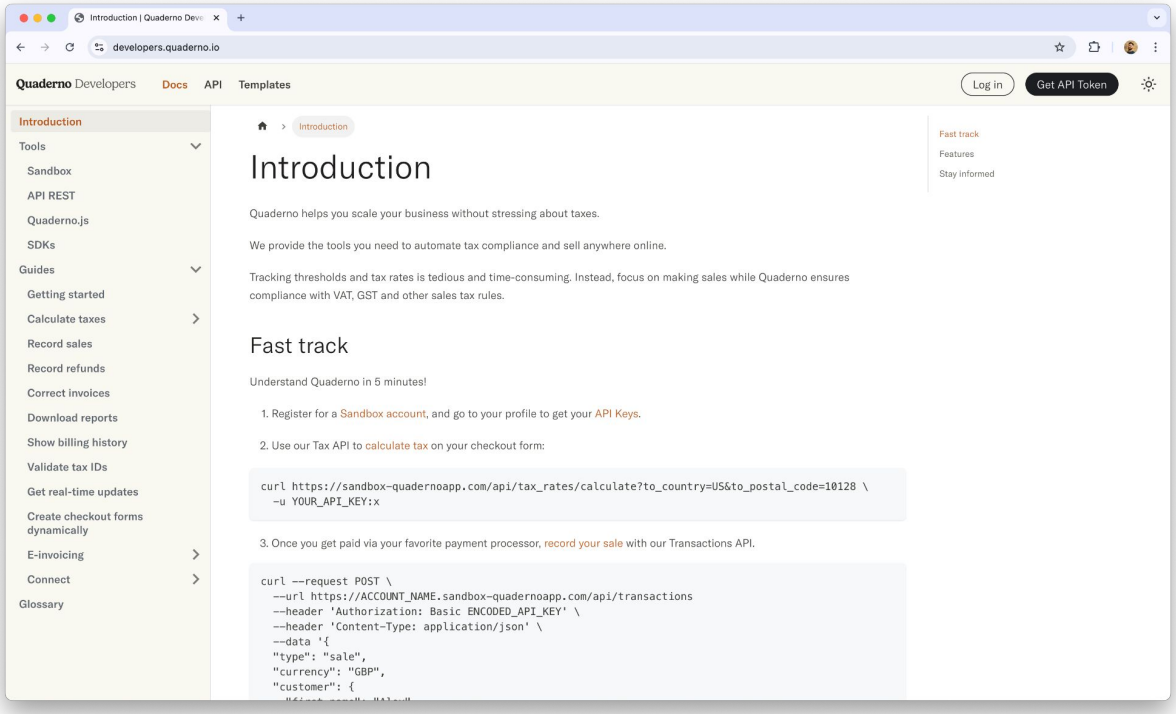
* Understand their needs and emotions when they start integrating.

Then, **write guides that help your users achieve their goals**.

Recommendation: Diátaxis framework (Daniele Procida)

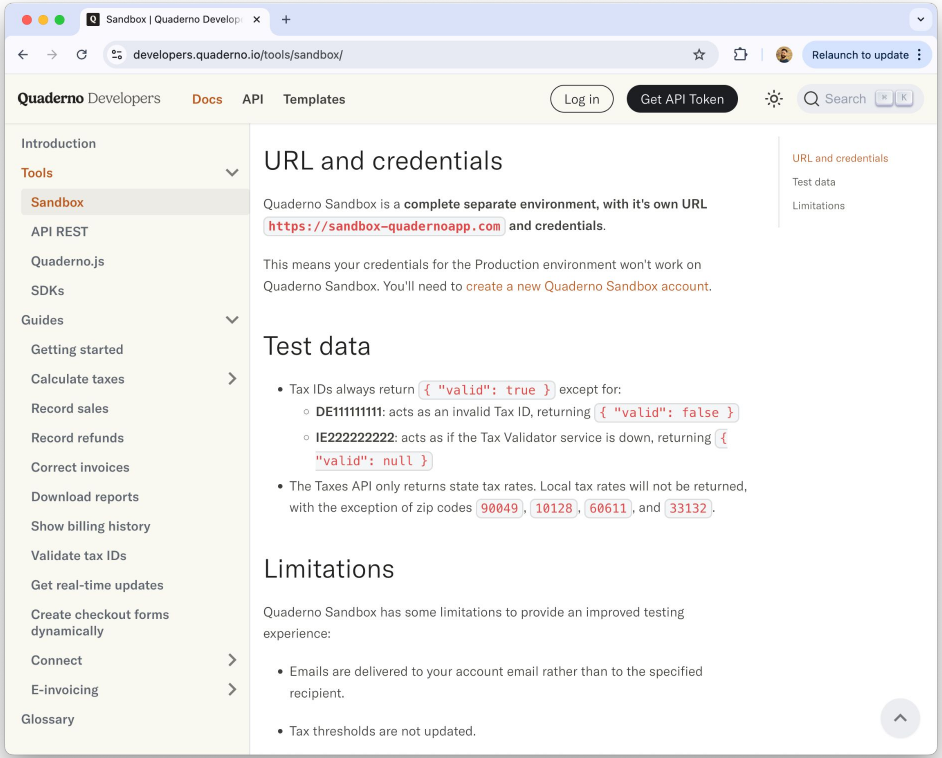


Example: Quaderno Developer's site



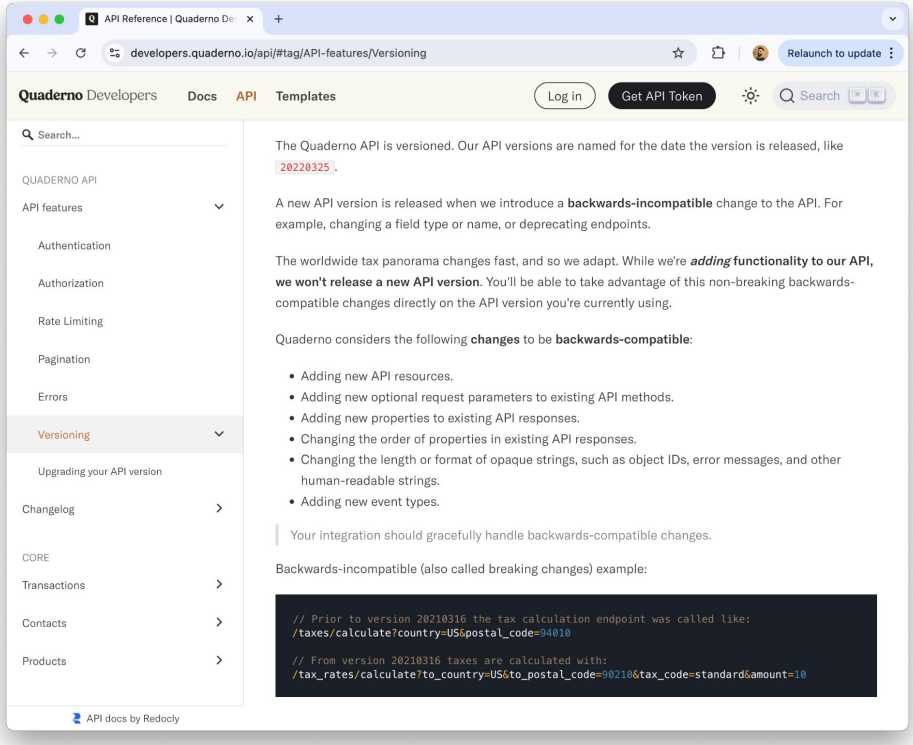
How-to guide opening, step by step

Developers value predictability



Testing environments matter

Developers value predictability



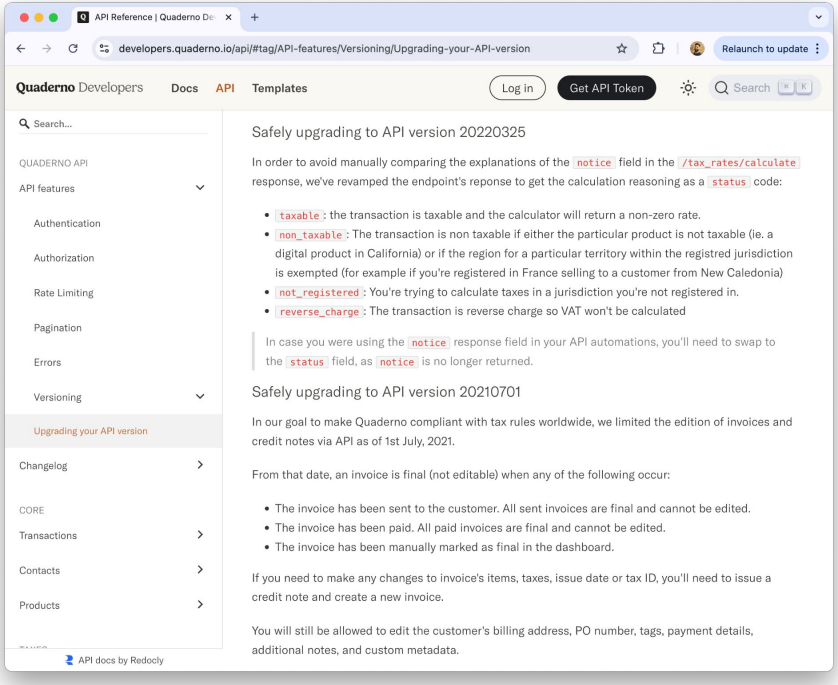
Publish a clear **versioning policy**

Developers value predictability

Communicate breaking changes

Publish **migration guides**

Keep a **Changelog**

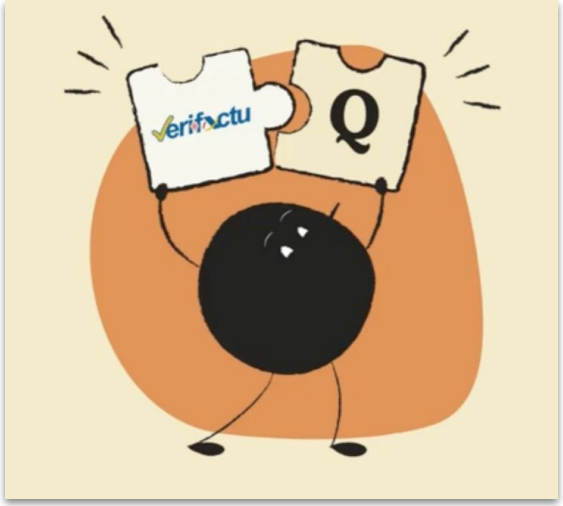


Example: Verifactu at Quaderno

Legal constraints forcing breaking changes.

Clear communication saved support time and user frustration:

* Webinars
* Emails to customer base
* Updating Dev guides and API reference
* Providing safe testing environment



Final thoughts: Documentation as an organism



*https://diataxis.fr/how-to-use-diataxis/#**complete-not-finished***

Documentation is not an afterthought,

it’s the API’s interface to the world.

Questions & Answers time!

Thanks

for your time!

 @jrubr  /javi-rubio  /jrub